

Uporedna analiza *database pattern*-a u mikroservisnoj arhitekturi

Autori:

Pešić Teodora IN1/2022

Todorović Marija IN13/2022

Lukić Nenad IN17/2022

Sančanin Srđan IN59/2022

Uvod

U početku su se softverski sistemi projektovali i razvijali kao jedna samostalna celina i ovaj pristup je bolje poznat kao monolitna arhitektura. Iako vrlo jednostavna za implementaciju, testiranje, pronalazak greški i isporuku, pokazala se kao nepraktična u slučaju velikih sistema. Za potrebe prevazilaska novonastalih izazova razvila se mikroservisna arhitektura, gde postoji skup nezavisnih servisa koji međusobno komuniciraju. Ovakav pristup omogućava veću skalabilnost, primenu različitih tehnologija i programskih jezika, lakše održavanje i nezavisno razvijanje pojedinačnih komponenti sistema. Međutim, pored brojnih prednosti, mikroservisna arhitektura donosi i određene izazove.

Jedan od osnovnih principa mikroservisnih aplikacija jeste autonomija servisa, što podrazumeva da svaki servis poseduje i upravlja sopstvenim podacima. Međutim, u realnim sistemima često postoji potreba za razmenom podataka između različitih servisa, što otvara pitanja konzistentnosti podataka.

Distribuirana priroda mikroservisa otežava primenu klasičnih transakcija i mehanizama za očuvanje konzistentnosti podataka. Zbog toga su razvijeni različiti *database pattern*-i koji nude rešenja za organizaciju skladištenja podataka i razmenu informacija između servisa.

Database pattern-i predstavljaju skup arhitektonskih rešenja za organizaciju, upravljanje i pristup podacima. Pravilnim izborom šablona moguće je unaprediti skalabilnost sistema, poboljšati performanse, pojednostaviti održavanje i obezbediti odgovarajući nivo konzistentnosti podataka. Među najpoznatijim pristupima izdvajaju se: *Database per Service*, *Shared Database*, *SAGA*, *CQRS* i *Event Sourcing*, pri čemu svaki od njih rešava navedene probleme na drugačiji način. Izbor odgovarajućeg šablona zavisi od specifičnih zahteva sistema i poslovnog domena.

Cilj ovog rada jeste analiza i poređenje dva često korišćena *database pattern*a u mikroservisnoj arhitekturi, *Shared Database* i *Database per Service*, kako bi se identifikovale njihove prednosti, nedostaci i odgovarajuće oblasti primene.

Database per Service pristup

Database per Service predstavlja jednu od najčešće korišćenih *data pattern*-a u mikroservisnoj arhitekturi. Osnovna ideja ovog pristupa jeste da svaki mikroservis ima svoju bazu podataka i da njenim podacima pristupaju isključivo komponente tog servisa. Na taj način servis ima potpunu kontrolu nad svojim podacima i nije direktno zavisn od drugih servisa i njihovih baza podataka.

S tim u vezi, kao jednu od najznačajnijih prednosti ovog pristupa izdvojila bih visok stepen autonomije koji dozvoljava menjanje strukture baze podataka, modela podataka ili način njihovog skladištenja bez potrebe za izmenama u drugim servisima. Ovakva nezavisnost smanjuje međusobnu povezanost komponenti sistema.

Još jedna značajna prednost jeste mogućnost nezavisnog skaliranja servisa i njihovih baza podataka u skladu sa stvarnim potrebama sistema. U praksi se neretko dešava da pojedini servisi imaju znatno veće opterećenje od ostalih. Zahvaljujući ovom obrascu, moguće je povećati resurse samo za one servise koji predstavljaju usko grlo, bez potrebe za skaliranjem celokupnog sistema kao u slučaju monolitnih aplikacija.

Takođe ovaj pristup otvara mogućnost korišćenja različitih tipova baza za različite servise, u zavisnosti od njihovih zahteva. Na primer, jedan servis može koristiti relacionu bazu podataka, dok drugi može koristiti dokumentnu ili grafsku bazu podataka. Ovakav pristup, poznat kao *Polyglot Persistence*.

Bitno je napomenuti i povećanu otpornost sistema na greške. Problemi koji nastanu u jednoj bazi ne moraju nužno dovesti do otkaza celokupnog sistema. U slučaju nedostupnosti jednog servisa ili njegove baze podataka, ostali servisi mogu normalno nastaviti sa radom, što doprinosi većoj pouzdanosti i dostupnosti aplikacije.

Pored brojnih prednosti, *Database per Service* uvodi i određene prepreke. Najveći problem predstavlja održavanje konzistentnosti podataka između više servisa. Kako nije moguće jednostavno koristiti klasične *ACID* transakcije često se primenjuju alternativni mehanizmi, kao što su *SAGA* obrazac i *event-driven* komunikacija, koji povećavaju složenost sistema i zahtevaju dodatne napore prilikom implementacije i testiranja.

Još jedan izazov odnosi se na izvršavanje upita koji zahtevaju podatke iz više servisa. U monolitnim sistemima ovakvi upiti se realizuju korišćenjem *JOIN* operacija nad jednom bazom, dok je kod *Database per Service* pristupa potrebno prikupiti podatke iz više izvora putem mrežne komunikacije. To može dovesti do povećanja latencije, složene implementacije i većeg opterećenja sistema.

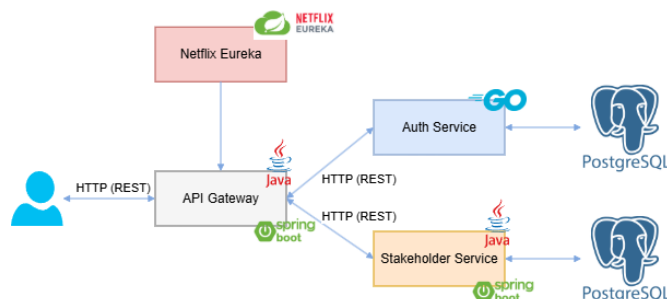
Sinhronizacija promena predstavlja zadatak za sebe. Ukoliko više servisa koristi povezane podatke, potrebno je obezbediti pouzdanu razmenu informacija kako bi svi servisi raspolagali ažurnim podacima. Greške u komunikaciji, kašnjenje poruka ili privremena nedostupnost servisa mogu dovesti do nekonzistentnog stanja sistema.

Database per Service obrazac najčešće se koristi u velikim distribuiranim sistemima kod kojih su skalabilnost, nezavisnost razvoja i jednostavnije održavanje od velikog značaja. Primeri primene mogu se pronaći u sistemima za elektronsku trgovinu, bankarskim aplikacijama, platformama za rezervacije i drugim poslovnim sistemima sa velikim brojem korisnika i visokim zahtevima za dostupnošću i skalabilnošću. *Database per Service* se danas smatra preporučenim obrascem. Ipak, njegova primena zahteva pažljivo rešavanje pitanja konzistentnosti podataka i komunikacije između servisa kako bi se obezbedilo pravilno funkcionisanje celokupnog sistema.

Primena *Database per Service* pristupa na primeru

Kako bismo prikazali kako izgleda primena samog “*Database Per Service*” pristupa odlučili smo se da koristimo arhitekturu prikazanu na slici br. 1. U pitanju je jednostavna aplikacija koja se sastoji od 2 mikroservisa sa njihovim odvojenim bazama i 2 dodatna mikroservisa koja igraju ulogu pristupnika i registra servisa:

- *auth-service* koji služi za upravljanje *JWT* tokenima i sesijama samih korisnika, omogućava funkcionalnosti logovanja i registracije i komunicira sa *PostgreSQL* bazom podataka,
- *stakeholder-service* koji služi za upravljanje i prikaz samih naloga korisnika i imaće značajnu ulogu kasnije vezanu za upotrebu SAGA obrasca i komunicira sa *PostgreSQL* bazom podataka,
- *API Gateway* koji je implementiran u programskom jeziku *Java* uz upotrebu *Spring Boot* radnog okvira i služi za rutiranje poziva ka samim mikroservisima i upravo igra ulogu pristupnika i
- *Eureka* servis koji obavlja funkciju registra servisa i daje informacije o lokaciji samih servisa pristupniku.



Slika br. 1: Prikaz mikroservisne arhitekture za *Database per Service* pristup

Specifičnosti šema baza podataka u *Database per Service* pristupu

Jedna od osobina na kojima upravo leži mikroservisna arhitektura jeste upravo mogućnost izbora između skupa tehnologija koji će biti korišćeni prilikom kreiranja samog servisa. *Database per Service* pristup nam osim te osobine donosi i slobodu da za svaki servis odaberemo bazu podataka koja će najviše odgovarati našem slučaju korišćenja (engl. *use case*). U ovom primeru, oba mikroservisa se oslanjaju na relacione baze podataka, specifično *PostgreSQL*, gde svaki mikroservis upravlja svojom bazom podataka.

users (Stakeholder Service)	
PK	id : BIGSERIAL
UQ	registry_id : BIGINT NOT NULL
UQ	username : VARCHAR(32) NOT NULL
UQ	email : VARCHAR(255) NOT NULL
	role : VARCHAR(32) NOT NULL
	first_name : VARCHAR(255)
	last_name : VARCHAR(255)
	profile_image : VARCHAR(255)
	biography : TEXT
	motto : VARCHAR(255)
	is_blocked : BOOLEAN NOT NULL DEFAULT false

users (Auth Service)	
PK	id : BIGSERIAL
UQ	username : VARCHAR(32) NOT NULL
UQ	email : VARCHAR(255) NOT NULL
	role : VARCHAR(32) NOT NULL
	password : TEXT NOT NULL

Slika br. 2: Prikaz šema baza podataka u servisima

Na slici br. 2 su prikazane kako same tabele koje predstavljaju zapravo 2 šeme baze podataka, što je uzrokovano jednostavnom prirodom samih servisa, koje se koriste u samim servisima. Pored same jednostavnosti samih šema baza podataka, polje *registry_id* tabele *users* bi u slučaju deljene baze podataka između servisa zapravo predstavljalo ograničenje stranog ključa koje referencira polje *id* iz tabele *registry*. U ovom slučaju, pošto nije moguće implementirati ograničenje stranog ključa između različitih baza podataka koje su podignute na odvojenim servisima, polje *registry_id* ne predstavlja ograničenje stranog ključa, ali još uvek predstavlja važnu vezu između samih servisa i moglo bi se koristiti prilikom operacija ažuriranja nad samim torkama.

Jasno se iz samih šema baze podataka vidi da tabele sadrže samo polja koja su im značajna za obavljanje poslovnih funkcionalnosti, odnosno u šemi baze podataka za *Auth Service* se nalaze svi značajni podaci koji će imati ulogu prilikom kreiranja *JWT* tokena i validaciju istih, dok će se u šemi baze podataka za *Stakeholder Service* nalaziti podaci koji su značajni za upravljanje korisničkim profilima.

Zbog odvojenosti samih mikroservisa, kao i činjenice da su im i baze podataka odvojene, odnosno da servisi ne dele bazu podataka, moguće je poraditi na skalabilnosti i dostupnosti same aplikacije, uz uvođenje do sada nepoznatih problema oko izvršavanja samih transakcija, u čemu će biti reči u narednim sekcijama.

Upotreba transakcija u mikroservisnoj arhitekturi

Transakcija je skup operacija koje želimo da izvršimo nad našim podacima i tipično one prate “sve ili ništa” pravilo, odnosno ili su sve operacije uspešno izvršile ili se vraćamo u prethodno konzistentno stanje. U monolitnim sistemima, o prirodi transakcija brine sistem za upravljanje bazom podataka. [1] Ako se transakcije izvršavaju u distribuiranim arhitekturama, tad se nazivaju distribuiranim transakcijama. Postoji više načina na koje je moguće obezbediti izvršenje transakcija u mikroservisnoj arhitekturi, pa samim tim i garantovati konzistentnost podataka, a to je upotrebom nekih od sledećih načina upravljanja samim transakcijama: *2PC* protokol, *SAGA* šablon (postoje orkestrirana i koreografisana verzija) i *3PC* protokol koji iako rešava mane prethodnog *2PC* protokola, uvodi dosta veću kompleksnost u sistem i stoga se jako retko koristi u produkciji.

2PC protokol

2PC protokol odnosno *2 Phase Commit* protokol predstavlja distribuirani algoritam koji upravlja procesima koji učestvuju u distribuiranoj transakciji. Predstavlja protokol konsenzusa zbog svoje osobine da se procesi dogovaraju u vezi obavljanja transakcija. Sastoji se od 2 faze kojima upravlja koordinator:

- faza glasanja u kojoj koordinator pita čvorove radnike da li mogu da izvrše svoj deo transakcije i radi se blokirajuće čekanje, dok se ne dobije odgovor od svakog servisa,
- *commit* faze koja se izvršava, ako su svi čvorovi procesa potvrdno glasali, odnosno na *rollback* samog izvršavanja, ako nisu svi potvrdno glasali.

Iako protokol obezbeđuje da izvršene transakcije ispunjavaju *ACID* principe, ujedno uvodi mane u samo izvršavanje transakcija zbog: povećanja same spregnutosti između mikroservisa koja se javlja usled blokirajućeg čekanja, sinhrono prirode izvršavanja samog protokola i zavisnosti od najsporijeg mikroservisa za nastavak ili izvršenje same transakcije. [2] Zbog svojih mana i same kompleksnosti implementacije i izvršavanja, ovaj protokol nije implementiran u okviru primera.

SAGA šablon

SAGA šablon predstavlja alternativni način upravljanja distribuiranim transakcijama i posebno je značajan u mikroservisnoj arhitekturi zbog dugovečnosti samih transakcija. Umesto da se atomičnost transakcije garantuje kroz jednu “sve ili ništa” transakciju, ovaj šablon omogućava da se transakcija razloži na niz manjih i nezavisnih podtransakcija koje se nazivaju lokalne transakcije. Svakom podtransakcijom upravlja poseban servis i one zajedno sačinjavaju ovaj šablon. Ako se lokalna transakcija ne izvrši uspešno, *SAGA* šablon izvršava niz kompenzacionih transakcija kako bi se sistem vratio u prethodno konzistentno stanje. Postoji 3 tipa transakcija: transakcije čije izvršavanje se može preokrenuti

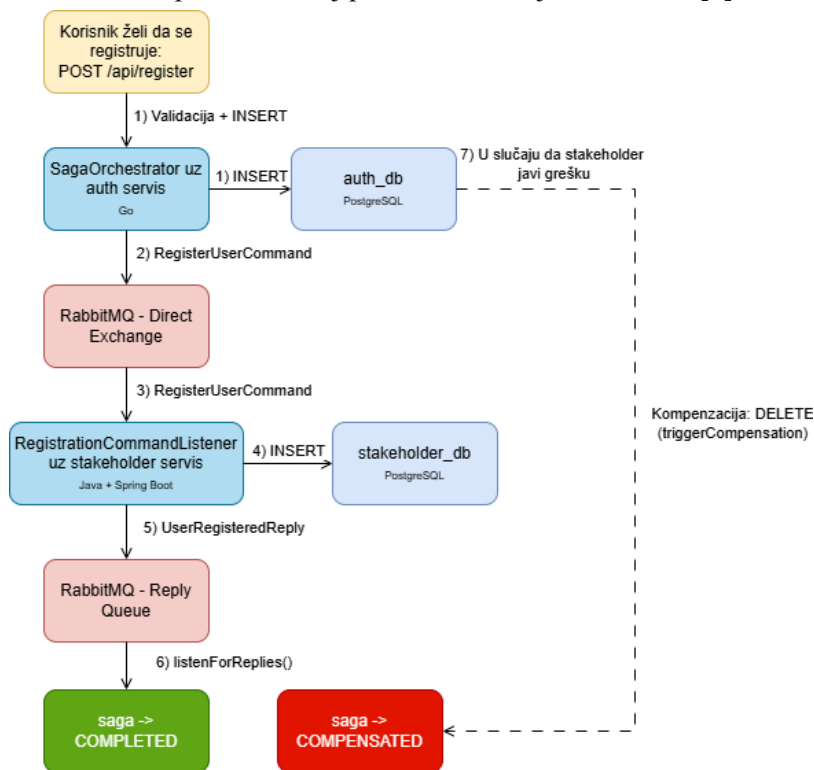
primenom transakcije sa suprotnim efektom (engl. *compensable transactions*), pivot transakcije i ponovno izvršive transakcije koje prate pivot transakcije i garantovano će se izvršiti. Postoje 2 načina implementacija *SAGA* šablona: orkestracija i koreografija. [1]

SAGA šablon za razliku od transakcija u nedistribuiranim sistemima, ili u odnosu na *2PC* protokol ne garantuje očuvanje *ACID* principa, već *BASE* principa:

- *Basically Available*, što se odnosi na to da će sistem baza podataka biti spreman da odgovori na zahteve korisnika, čak iako ne može da garantuje pristup svim podacima. Baza podataka će možda doživeti periode nedostupnosti, ali treba da bude dizajnirana da su ti periodi što kraći i da omogući brz oporavak od pada. [3]
- *Soft State*, što se odnosi na to da se stanje baze podataka može menjati tokom vremena, čak i bez korisnikovog uticaja, već se dešava kao rezultat pozadinskih procesa, ažuriranja podataka i drugih faktora. Baza podataka treba da ovo podrži uz garantovanje da se neće javiti korupcija podataka ili njihov gubitak. [3]
- *Eventual Consistency* se odnosi na eventualnu konzistentnost podataka u samoj bazi podataka. Drugim rečima, baza podataka bi trebala eventualno da konvergira ka konzistentnom stanju, čak i ako joj je potrebno vreme za propagaciju ažuriranja nad podacima. Ovo je direktan kontrast u odnosu na odmah dostupnu konzistentnost kod *ACID* transakcija. [3]

Orkestrirani *SAGA* šablon

U orkestriranoj verziji *SAGA* šablona, centralni orkestrator (koordinator) upravlja celokupnim tokom transakcije. Orkestrator šalje komande svakom servisu kako bi se izvršile lokalne transakcije i takođe upravlja neuspesima slanjem kompenzacionih transakcija. U slučaju pada bilo koje podtransakcije, orkestrator šalje komande kako bi se poništio uticaj prethodno načinjenih koraka. [1]

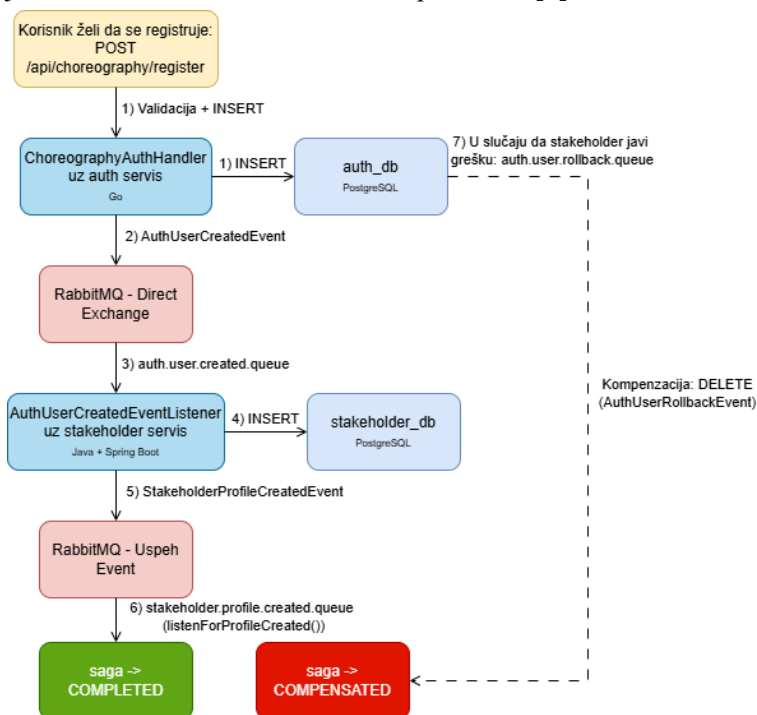


Slika br. 3: Prikaz implementacije orkestriranog *SAGA* šablona

U našem primeru, orkestrirani *SAGA* šablon je primenjen nad procesom registracije novog korisnika u sistem. Primena *SAGA* šablona je bila moguća zbog same osobine registracije da se korisnik mora uneti u 2 odvojene baze podataka u samom sistemu. Postoji i odvojeni *API endpoint* koji služi za dobijanje informacije o statusu izvršenosti prilikom kreiranja naloga.

Koreografisani *SAGA* šablon

U koreografisanoj verziji *SAGA* šablona, svaka lokalna transakcija objavljuje događaj (engl. *event*) koji okidaju lokalne transakcije u narednim servisima. U suštini, ne postoji centralni koordinatorski koji govori učesnicima šta treba da rade. U slučaju pada na bilo kom koraku, svaki servis mora da izvrši kompenzacione transakcije kako bi se sistem vratio na stare promene. [1]



Slika br. 4: Prikaz implementacije koreografisanog *SAGA* šablona

U našem primeru, koreografisani *SAGA* šablon je primenjen nad procesom registracije novog korisnika u sistem. Primena *SAGA* šablona je bila moguća zbog same osobine registracije da se korisnik mora uneti u 2 odvojene baze podataka u samom sistemu. Postoji i odvojeni *API endpoint* koji služi za dobijanje informacije o statusu izvršenosti prilikom kreiranja naloga.

Osobine *Database per Service* pristupa

Kao što smo ranije i videli, ovaj pristup nam omogućava da svaki mikroservis ima svoju zasebnu bazu podataka. Umesto toga da svi mikroservisi dele jednu istu centralnu bazu podataka, svaki mikroservis ima i brine o svojim podacima u svojoj bazi podataka koja mu najviše odgovara. Glavni ciljevi ovakvog pristupa su: [5]

- Nezavisnost baza podataka - promene šeme baze podataka u jednom servisu ne utiče na ostale.
- Enkapsulacija - pristup je moguć samo preko *API endpoint*-a, a ne upotrebom direktnih poziva nad bazom podataka.

- Izbor tehnologija - svaki tim može da odabere bazu podataka koja mu najviše odgovara.
- Skalabilnost - baze podataka se nezavisno skaliraju.
- Otpornost - pad baze podataka u jednom servisu ne uzrokuje pad celokupnog sistema.

Zbog navedenih ciljeva samih mikroservisa, kao i činjenice da su im i baze podataka odvojene, moguće je značajno poraditi na skalabilnosti same aplikacije dodavanjem više instanci istog servisa koji će slušati nad istom bazom podataka kao i prva nastala instanca, čime se ne opterećuje jedna zajednička baza podataka, ali takođe možemo raditi i na dostupnosti same aplikacije upravo zbog dodatne otpornosti.

API Gateway bi u slučaju postojanja više instanci istog servisa vršio balansiranje opterećenja (engl. *load balancing*) sa klijentske strane upotrebom informacija dobijenih iz *Eureka* registra servisa.

Dostupnost same aplikacije se povećava zbog otpornosti na otkaze koja je upravo dobijena odvajanjem samih baza podataka, odnosno, ako jedan servis ili njegova baza podataka padne, svi ostali servisi će moći da nastavu da budu dostupni korisnicima i da odgovaraju na njihove zahteve koji su vezani za već dostupne servise, dok će se za zahteve koji su vezani za nedostupni servis čekati do njegovog oporavka.

Glavni ciljevi ovakvog pristupa značajno utiču i daju nam njegove prednosti kao što su: niska spregnutost između servisa, nezavisno skaliranje, autonomija servisa, izolacija greške odnosno pada i sloboda biranja baze podataka za svaki servis zasebno. [5] Ovo sve sa sobom povlači i mane, od čega je najznačajnije izdvojiti kompleksnost i cenu samog razvoja.

Literatura

- [1] *Difference Between Two-Phase Commit and Saga Pattern*:
<https://www.baeldung.com/cs/two-phase-commit-vs-saga-pattern>
- [2] *Two-phase commit protocol*: https://en.wikipedia.org/wiki/Two-phase_commit_protocol
- [3] *BASE properties*: <https://www.geeksforgeeks.org/dbms/base-properties-in-dbms/>
- [4] *Should I use one or Multiple Databases across the same microservice?*:
https://stackoverflow.com/questions/59397689/should-i-use-one-or-multiple-databases-across-the-same-microservice?st_source=ai_mode
- [5] *Microservices Patterns : Database Per Service Pattern*:
<https://blog.singhabhinav.in/microservices-patterns-database-per-service-pattern-44e8e2df4612>